InvertedBlock Indexing for Learned Sparse MIPS

Roger Canal Estrada*

Universitat Politècnica de Catalunya (UPC)
Facultat d'Informàtica de Barcelona (FIB)
roger.canal02@gmail.com

Abstract. Learned sparse representations (LSRs), such as SPLADE, preserve lexical interpretability but incur high maximum inner product search (MIPS) latency in very high-dimensional sparse spaces. This paper presents an `InvertedBlock` indexing architecture with two phases: a static index partitioned through document clustering and a parameterized search based on coordinate-at-a-time (CaaT) traversal. The static phase controls candidate coverage, while the dynamic phase manages the latency–recall trade-off through runtime constraints. On Natural Questions (2.68 million documents and 30,522 dimensions), the system achieves Recall@30 ≥ 0.90 at an average latency of approximately 40 ms per query.

Keywords: Learned Sparse Retrieval · Approximate Nearest Neighbor Search · Dynamic Pruning · Inverted Indexing

1 Introduction

LSR models map semantics into vocabulary space, keeping retrieval sparse and interpretable [3]. However, they can assign important weights to tokens with no apparent semantic meaning, including punctuation. Classical lexical systems often discard such tokens, but doing so in LSR can alter rankings—a counter-intuitive behavior known as *wacky weights* [4,5]. More query dimensions must therefore be traversed, weakening WAND-like and document-at-a-time (DaaT) pruning because early partial-score bounds become less decisive [2].

This paper investigates a custom partitioned inverted data structure for this traversal regime. Posting lists are divided into cluster-aligned `InvertedBlock`s whose exact summary vectors bound clusters before documents are scored. Static candidate organization is separated from dynamic traversal constraints to control the latency–recall trade-off at query time.

The paper makes three contributions: a configurable static *Inverted Block Index* that preserves candidate coverage; exact-maximum summaries that make block bounds safe for aggressive skipping; and an operating point above Recall@30 = 0.90. Clustered postings and summary vectors are also explored in SEISMIC [6], the closest prior design point for this investigation.

* Corresponding author ORCID: [0009−0008−8198−6372]

2 System Architecture

The system is built around a custom partitioned inverted index designed to support efficient approximate maximum inner product search over sparse vectors. The data structure organizes document information so that the search engine can quickly identify the documents most similar to an incoming query without scoring every document individually.

The pipeline has two phases: static `InvertedBlock` construction (Section 2.1) and parameterized query-time search (Section 2.2).

2.1 Static Index Phase

The static index is built in three steps. First, sparse K-means assigns each document to exactly one cluster. Second, for every cluster and vocabulary term, the index computes exact maximum term weights to build dense summary vectors. Third, for each term, it retains only the top-nb clusters and then the top-nd documents per retained cluster, producing the final `InvertedBlock` lists used at query time.

Document Clustering Step. The method partitions the document set $\mathcal{D} = \{d_i\}_{i=1}^{N}$ into K clusters by a sparse K-means objective:

$$\max_{\{\mathcal{C}_k\},\{\mu_k\}} \sum_{k=1}^{K} \sum_{d_i \in \mathcal{C}_k} \langle d_i, \mu_k \rangle, \tag{1}$$

with assignment step

$$a(i) = \arg \max_{k \in \{1,\dots,K\}} \langle d_i, \mu_k \rangle, \tag{2}$$

and centroid update

$$\mu_k \leftarrow \frac{\sum_{d_i \in \mathcal{C}_k} d_i}{\left\| \sum_{d_i \in \mathcal{C}_k} d_i \right\|_2}. \tag{3}$$

Both operations are computed in sparse form. This is a *hard clustering*: each document d_i is assigned to exactly one cluster $\mathcal{C}_{a(i)}$, and no document belongs to more than one cluster.

Partitioned Inverted Index Construction. The index maps each concept t to a list of `InvertedBlock`s. Each block stores one cluster ID and truncated document–weight pairs. For each cluster c, the method also builds a dense summary vector $S_c \in \mathbb{R}^{|V|}$ with

$$S_c[t] = \max_{d \in \mathcal{C}_c} d[t]. \tag{4}$$

Cluster-level summary vectors are also used in SEISMIC [6]; here, each component is the exact maximum weight of concept t within cluster c.

Algorithm 1 Static Index Construction with Exact-Maximum Summaries

Require: Sparse documents, cluster assignments, nb, nd
Ensure: Partitioned index I and summary vectors S
for each cluster c **do**
  initialize dense summary $S_c \leftarrow 0$
  for each document $d \in \mathcal{C}_c$ **do**
    for each non-zero (t, w) in d **do**
      $S_c[t] \leftarrow \max(S_c[t], w)$
    end for
  end for
end for
for each concept t **do**
  aggregate candidate docs by cluster and concept weight
  keep top-nb clusters by accumulated weight on t
  for each retained cluster c **do**
    keep top-nd docs by weight on t
    append `InvertedBlock`$(c, doc\_ids, weights)$ to $I[t]$
  end for
end for
return I, S

For each concept t, the index keeps at most nb cluster blocks and at most nd documents per retained block. Exact maxima make the cluster upper bound mathematically valid because it is never underestimated.

$$UB_c(q) = \sum_{t \in \text{supp}(q)} q[t] \cdot S_c[t] \tag{5}$$

This property prevents unsafe skipping: if the summaries underestimated term weights, the system could discard clusters containing true top-k documents.

2.2 Search Phase

Coordinate-at-a-Time (CaaT) Traversal. The search engine processes incoming sparse queries over the static index using CaaT traversal. [1] Algorithm 2 formalizes the CaaT loop.

The search engine exposes four hyperparameters that restrict traversal:

1. `heap_factor`: a relaxation multiplier that controls skip confidence.
2. `max_query_terms` (mqt): truncates low-signal query tail coordinates.
3. `max_search_blocks` (msb): limits per-coordinate block breadth online.
4. `max_docs_to_visit` (md): a hard computational budget that bounds worst-case latency.

The algorithm parses the non-zero query coordinates, sorts them by descending weight, and traverses the inverted list for each coordinate in turn. It thus processes the strongest query evidence first, applies the query-coordinate cap when requested, and uses block upper bounds to skip unpromising regions before consuming the document budget.

[1] CaaT is preferred over classical document-at-a-time (DaaT) traversal [2] because it visits the highest-signal query coordinates first, rapidly raising the min-heap threshold. A high threshold strengthens subsequent block-skipping decisions. By contrast, DaaT mixes low- and high-signal evidence across documents before the threshold stabilizes, reducing early pruning power.

Algorithm 2 Parameterized CaaT Search

Require: Query q, index I, summaries S, top-k, Max Query Terms mqt, Max Search Blocks msb, $heap\_factor$, Max Docs md
Ensure: Top-k result heap H

$Q \leftarrow$ non-zero query coordinates sorted by weight descending
if $mqt > 0$ **then**
    truncate Q to first mqt terms
end if
compute $UB_c(q)$ for all clusters c
initialize min-heap H, visited set, $docs\_examined \leftarrow 0$
for each term $t \in Q$ **do**
    $blocks\_seen \leftarrow 0$
    for each block $b \in I[t]$ **do**
        if $msb > 0$ and $blocks\_seen \geq msb$ **then**
            break
        end if
        $blocks\_seen \leftarrow blocks\_seen + 1$
        if $UB_{b.cluster\_id}(q) < H.\min() \cdot heap\_factor$ **then**
            continue
        end if
        for each doc d in b **do**
            if d not visited **then**
                score $\leftarrow \langle q, d \rangle$ and update H
                mark visited; $docs\_examined \leftarrow docs\_examined + 1$
                if $md > 0$ and $docs\_examined \geq md$ **then**
                    return H
                end if
            end if
        end for
    end for
end for
return H

Aggressive Dynamic Pruning Heuristics. The pruning stage rejects blocks by the inequality:

$$UB_c(q) < H.\min() \cdot heap\_factor. \tag{6}$$

If the inequality holds, the block cannot improve the current top-k under the selected confidence multiplier. This rule replaces expensive document-level scoring with $O(1)$ floating-point comparisons at block granularity.

3 Experimental Evaluation

3.1 Experimental Setup

The evaluation uses the SISAP 2026 Task 3 dataset [1]: Natural Questions (NQ), with 2.68 million documents encoded as SPLADE sparse vectors in 30,522 dimensions. Recall@30 is measured against the provided exact k-NN ground truth, and average query latency is reported. [2]

Experimental methodology. A first study varied the index-building parameters to characterize their effects on retrieval quality, latency, and index capacity. Then the search parameters were studied to determine their behavior and identify effective operating points. From the index experiments, nine representative configurations were selected for the search experiments.

[2] Implementation repository: `https://github.com/channelEs/seed_42-SISAP-26`.

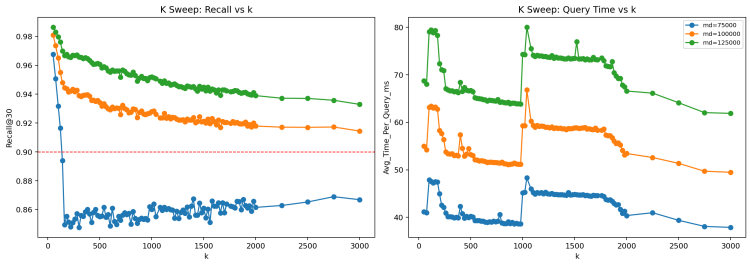

Fig. 1. Recall@30 and average query time as a function of K for three md values ($md = [50{,}000, 100{,}000, 150{,}000]$).

Hardware. All runs execute inside a Docker container constrained to 8 virtual CPUs and 24 GB RAM with swap disabled. The host machine is an AMD Ryzen 5 5600X (6 cores / 12 logical processors, base 3.70 GHz, 32 MB L3 cache). Minor rerun variation can still arise from runtime details such as thread scheduling and OpenMP placement (for example `OMP_NUM_THREADS`); therefore, trends and operating ranges are reported rather than over-interpreting a single point estimate.

3.2 Index Building Phase Experimentation

To study the build-time parameters, the search configuration was held fixed. The resulting experiments isolate how the static index configuration shapes the attainable recall under a common search regime. Because postings omitted during index construction by Alg. 1 cannot be recovered at query time, these sweeps reveal how strongly each static configuration constrains later search. The goal is therefore to select index configurations for the search-phase experiments that are both efficient and sufficiently permissive.

Effect of Cluster Count K. A general sweep of K was performed to understand the effect of cluster granularity on recall and latency. The experiment used three document-budget (md) values to reveal how the static index interacts with increasingly permissive search budgets. Figure 1 summarizes the trend.

Recall generally decreases as K increases, whereas latency remains comparatively stable. The sharpest change occurs at very low K values (approximately 50–140): recall declines markedly in all three scenarios without a corresponding latency change. This result suggests that fewer, larger clusters allow the remaining recall-oriented parameters to retain candidates more effectively. Beyond this low-K region, the behavior stabilizes and cluster count has a smaller effect on recall.

Effect of `InvertedBlock` Parameters (nb and nd). An additional general sweep of nb and nd was performed to understand their effects on index construction. In this experiment, md was fixed at 150,000, a high value chosen to expose the effects of the build parameters. Figure 2 summarizes the recall surface.

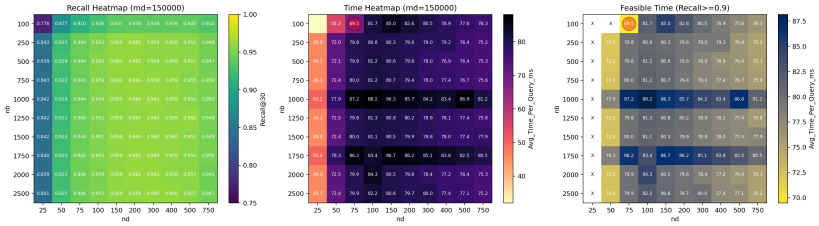

Fig. 2. Recall@30 heatmaps over nb and nd at $md = 150{,}000$.

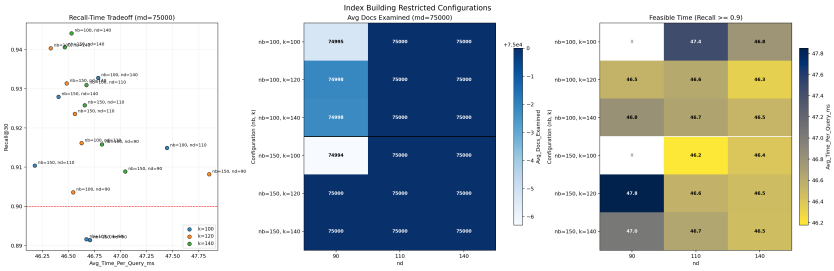

Fig. 3. Results of the restricted index-building experiments combining low K and nb/nd values.

The results confirm two patterns. First, increasing nd from very low values, such as 25, to the 150–400 range substantially improves recall because each retained cluster block contributes more document evidence. Second, increasing nb beyond the medium range yields smaller and less stable gains, which indicates diminishing returns once the most informative clusters are already retained. These results justify fixing a moderately permissive nb/nd setting for the search-phase experiments rather than pushing either parameter to its maximum.

Index Building Conclusions. Based on the K and nb/nd results, a narrower range combining low K with restricted nb/nd values was tested. The objective was to construct an efficient data structure that retains as much useful information as possible while minimizing index size. Figure 3 shows the results of these restricted experiments.

The results concentrate near Recall@30 = 0.90 while improving search performance. The average-documents-examined plot is particularly informative: at $nd = 90$, the search does not exhaust the maximum document budget. Increasing nd to 110 or 140 therefore enlarges the data structure only moderately while preserving additional candidates.

Based on these results, the following nine combinations were selected for the search-parameter experiments:

I-1: $(K, nb, nd) = (100, 150, 110)$, I-2: $(120, 100, 140)$, I-3: $(100, 150, 140)$, I-4: $(500, 500, 400)$, I-5: $(500, 750, 75)$, I-6: $(1000, 500, 400)$, I-7: $(1000, 100, 750)$, I-8: $(1500, 500, 400)$, and I-9: $(1500, 100, 750)$.

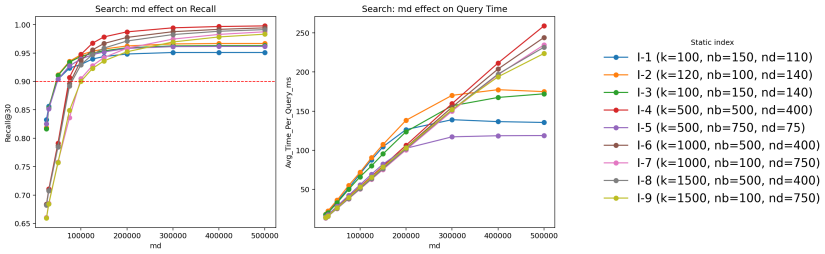

Fig. 4. Effect of *md* on Recall@30 and query latency across the nine static index configurations (I-1 to I-9).

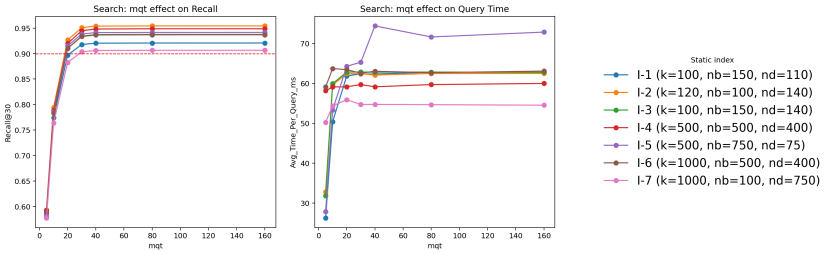

Fig. 5. Effect of *mqt* on Recall@30 and query latency across the nine static index configurations (I-1 to I-7).

3.3 Search Parameterization and Latency Optimization

With the nine fixed index configurations (I-1 to I-9), the search parameters are swept one at a time under a common baseline. For the *md* sweep, $mqt = 0$ and $msb = 150$; for the *mqt* and *msb* sweeps, $md = 100{,}000$. In all three experiments, `heap_factor` $= 0.05$. Algorithm 2 governs all four controls.

Maximum Number of Documents (*md*). The first and most important search constraint is the *md* parameter. Figure 4 shows how its value affects performance across index structures.

Across I-1 to I-9, recall grows monotonically with *md*, while latency generally increases and then tends to flatten once each index approaches its candidate ceiling. The first operating point with Recall@30 ≥ 0.90 appears between $md = 50{,}000$ and $md = 100{,}000$, depending on index compactness: compact settings such as I-1, I-2, I-3, and I-5 already cross 0.90 at $md = 50{,}000$ (for example, I-2 reaches 0.9118 at 35.9 ms/query), medium settings such as I-4 cross at $md = 75{,}000$ (0.9071 at 40.0 ms/query), and the most restrictive large-K settings (I-6 to I-9) require $md = 100{,}000$ (for example, I-9 reaches 0.9003 at 52.7 ms/query). These results confirm that *md* is the dominant recall–latency control: it sets the main quality frontier and provides a direct way to trade effectiveness for runtime.

Maximum Query Terms (*mqt*). The *mqt* parameter limits how many of the highest-weighted query coordinates are *included* in traversal. Figure 5 shows the sweep results.

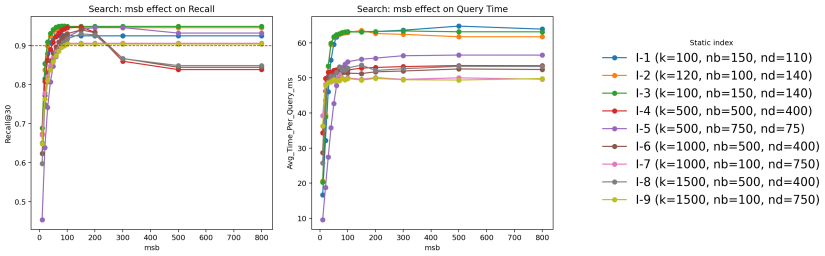

Fig. 6. Effect of *msb* on Recall@30 and query latency across the nine static index configurations (I-1 to I-9).

The completed *mqt* traces (I-1 to I-7) show a two-stage behavior. From very small caps ($mqt = 5$ or 10) to about $mqt = 20$–40, recall rises sharply because too many informative coordinates are being truncated at low *mqt* (for example, I-2 increases from 0.5929 at $mqt = 5$ to 0.9265 at $mqt = 20$). Beyond this region, gains become marginal: from $mqt = 40$ to 160, recall changes are tiny (typically below +0.001) and latency is nearly flat (roughly within about ±1 ms in the denser indexes). In practice, *mqt* should therefore be treated as a secondary tuning knob once a sufficient term budget is reached, rather than as a primary recall control.

Maximum Search Blocks (*msb*). The *msb* parameter limits the number of blocks evaluated per query coordinate. Figure 6 shows the results.

Unlike *mqt*, *msb* is a first-order search control across all nine index configurations. Increasing *msb* from 10 to 150 produces large recall gains in every case (about +0.234 to +0.487 absolute), with latency increases that depend on index compactness (about +10 to +46 ms/query). Around $msb \approx 150$, most traces saturate: in I-1, I-2, I-3, I-7, and I-9, recall is essentially unchanged from 150 up to 800. For denser settings (I-4, I-6, I-8), pushing much further can even reduce recall slightly while adding latency, indicating over-expansion into weaker blocks. These results support $msb \approx 150$ as a robust operating point that recovers most available quality at controlled runtime cost.

Heap Factor. Across the tested configurations, `heap_factor` has only a minor impact compared with the other hyperparameters. Therefore, `heap_factor` is fixed at 0.05 as a stable default, and attention is focused on the parameters that more strongly constrain the search.

4 Conclusion

This investigation shows that the proposed `InvertedBlock` architecture is a practical and robust data structure for approximate MIPS over learned sparse representations. The index-building study identifies a compact but sufficiently permissive family of configurations that preserves quality while avoiding unnecessary index growth. This analysis yields the nine selected index structures (I-1

to I-9) used in the search phase. In this regime, the structures consistently sustain high retrieval quality and achieve Recall@30 above 0.90 with practical query times.

The search experiments reveal a clear control hierarchy over this data structure: *md* defines the primary global recall–latency frontier, *msb* provides the main secondary recovery control with strong gains up to a stable saturation region, and *mqt* acts as a finer efficiency knob once enough high-weight query terms are retained. Together, these results confirm that the selected `InvertedBlock` design is not only mathematically safe through exact-maximum summaries, but also operationally tunable, allowing predictable quality–latency trade-offs under realistic runtime budgets.

Future work should evaluate this structure on broader workloads and over larger tuning spaces to quantify its full potential and limitations. Two immediate directions are automated search-parameter selection based on query-density statistics and memory-aligned forward-index layouts that accelerate sequential access during exact sparse dot products.

Acknowledgments. The author thanks Eneko Sabaté, another SISAP 2026 Indexing Challenge author, for encouraging participation and providing theoretical support, and UPC professor Amalia Duch-Brown for her support and the opportunity. The ALBCOM research group at Universitat Politècnica de Catalunya (UPC) partially funded conference attendance.

Disclosure of Interests. The author has no competing interests to declare that are relevant to the content of this article.

References

1. Aumüller, M., Fröbe, M., Mic, V., Tellez, E.S.: Overview of the SISAP 2026 Indexing Challenge: k-NN graphs, LLM workloads, and sparse embeddings. In: *Similarity Search and Applications: 19th International Conference, SISAP 2026, Proceedings*. Springer (2026)
2. Broder, A. Z., Carmel, D., Herscovici, M., Soffer, A., Zien, J.: Efficient query evaluation using a two-level retrieval process. In: *Proceedings of the 12th International Conference on Information and Knowledge Management (CIKM)* (2003)
3. Formal, T., Piwowarski, B., Clinchant, S.: SPLADE: Sparse lexical and expansion model for first stage ranking. In: *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval* (2021)
4. Mackenzie, J., Trotman, A., Lin, J.: Wacky weights in learned sparse representations and the revenge of score-at-a-time query evaluation. *arXiv preprint arXiv:2110.11540* (2021)
5. Roşca, G., Zamani, H., Makhervaks, M., et al.: Understanding wacky weights: A dissection of SPLADE's learned term importance. *arXiv preprint arXiv:2605.19628* (2026)
6. Bruch, S., Nardini, F. M., Rulli, C., Venturini, R.: Efficient inverted indexes for approximate retrieval over learned sparse representations. In: *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (2024)